

Notes on Chapter 13 – Numbers

Norman Lim

2025-07-27

Prerequisites

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.2      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.1.0
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(nycflights13)
```

Making numbers

- parsing strings into numbers using the `readr` package:

```
x <- c("1.2", "5.6", "1e6")
```

```
parse_double(x)
```

```
## [1] 1.2e+00 5.6e+00 1.0e+06
```

- parsing when the intended string contains non-numeric text that needs to be ignored:

```
x <- c("$1,234", "USD 3,513", "59%")
```

```
parse_number(x)
```

```
## [1] 1234 3513 59
```

Counts

- Using `count` for quick exploration and checks:

```
flights |> count(dest)
```

```
## # A tibble: 105 x 2
##   dest      n
##   <chr> <int>
## 1 ABQ    254
## 2 ACK    265
```

```
## 3 ALB      439
## 4 ANC        8
## 5 ATL    17215
## 6 AUS    2439
## 7 AVL     275
## 8 BDL     443
## 9 BGR     375
## 10 BHM     297
## # i 95 more rows
```

- Using count to see the most common values:

```
flights |> count(dest, sort = TRUE)
```

```
## # A tibble: 105 x 2
##   dest      n
##   <chr> <int>
## 1 ORD   17283
## 2 ATL   17215
## 3 LAX   16174
## 4 BOS   15508
## 5 MCO   14082
## 6 CLT   14064
## 7 SFO   13331
## 8 FLL   12055
## 9 MIA   11728
## 10 DCA    9705
## # i 95 more rows
```

- To see all the results, we can add |> View() or |> print(n = Inf)
- Doing count() “by hand” using group_by(), summarize(), and n(). (Useful if you want to compute for other summaries too.)

```
flights |>
  group_by(dest) |>
  summarize(
    n = n(),
    delay = mean(arr_delay, na.rm = TRUE)
  )
```

```
## # A tibble: 105 x 3
##   dest      n delay
##   <chr> <int> <dbl>
## 1 ABQ    254  4.38
## 2 ACK    265  4.85
## 3 ALB    439 14.4
## 4 ANC      8 -2.5
## 5 ATL   17215 11.3
## 6 AUS   2439  6.02
## 7 AVL    275  8.00
## 8 BDL    443  7.05
## 9 BGR    375  8.03
## 10 BHM    297 16.9
## # i 95 more rows
```

- Counting distinct values using n_distinct():

```
flights |>
  group_by(dest) |>
  summarize(carriers= n_distinct(carrier)) |>
  arrange(desc(carriers))
```

```
## # A tibble: 105 x 2
##   dest carriers
##   <chr>     <int>
## 1 ATL         7
## 2 BOS         7
## 3 CLT         7
## 4 ORD         7
## 5 TPA         7
## 6 AUS         6
## 7 DCA         6
## 8 DTW         6
## 9 IAD         6
## 10 MSP        6
## # i 95 more rows
```

- Getting a weighted count by summing:

```
flights |>
  group_by(tailnum) |>
  summarize(miles = sum(distance))
```

```
## # A tibble: 4,044 x 2
##   tailnum miles
##   <chr>     <dbl>
## 1 D942DN    3418
## 2 NOEGMQ   250866
## 3 N10156   115966
## 4 N102UW    25722
## 5 N103US    24619
## 6 N104UW    25157
## 7 N10575   150194
## 8 N105UW    23618
## 9 N107US    21677
## 10 N108UW    32070
## # i 4,034 more rows
```

- Getting a weighted count using the `wt` argument to `count()`:

```
flights |> count(tailnum, wt = distance)
```

```
## # A tibble: 4,044 x 2
##   tailnum      n
##   <chr>     <dbl>
## 1 D942DN    3418
## 2 NOEGMQ   250866
## 3 N10156   115966
## 4 N102UW    25722
## 5 N103US    24619
## 6 N104UW    25157
## 7 N10575   150194
## 8 N105UW    23618
```

```
## 9 N107US 21677
## 10 N108UW 32070
## # i 4,034 more rows
```

- Counting the number of missing values by combining `sum()` and `is.na()`:

```
flights |>
  group_by(dest) |>
  summarize(n_cancelled = sum(is.na(dep_time)))
```

```
## # A tibble: 105 x 2
##   dest n_cancelled
##   <chr>      <int>
## 1 ABQ         0
## 2 ACK         0
## 3 ALB        20
## 4 ANC         0
## 5 ATL       317
## 6 AUS        21
## 7 AVL        12
## 8 BDL        31
## 9 BGR        15
## 10 BHM       25
## # i 95 more rows
```

Exercises

1. How can you use `count()` to count the number of rows with a missing value for a given variable?

Ans. Combine `count()` with `is.na()`.

```
flights |>
  count(is.na(dep_time)) |>
  rename(
    is_na = `is.na(dep_time)`,
    count = n
  ) |>
  mutate(is_na = if_else(is_na == FALSE, "not missing", "missing"))
```

```
## # A tibble: 2 x 2
##   is_na      count
##   <chr>      <int>
## 1 not missing 328521
## 2 missing    8255
```

2. Expand the following calls to `count()` to instead use `group_by()` `summarize()`, and `arrange()`:

1. `flights |> count(dest, sort = TRUE)`
2. `flights |> count(tailnum, wt = distance)`

Ans.

for No. 1

```
flights |>
  group_by(dest) |>
  summarize(
    n = n()
  )
```

```

) |>
  arrange(desc(n))

## # A tibble: 105 x 2
##   dest      n
##   <chr> <int>
## 1 ORD    17283
## 2 ATL    17215
## 3 LAX    16174
## 4 BOS    15508
## 5 MCO    14082
## 6 CLT    14064
## 7 SFO    13331
## 8 FLL    12055
## 9 MIA    11728
## 10 DCA     9705
## # i 95 more rows

for No. 2
flights |>
  group_by(tailnum) |>
  summarize(miles = sum(distance))

## # A tibble: 4,044 x 2
##   tailnum miles
##   <chr>   <dbl>
## 1 D942DN    3418
## 2 NOEGMQ  250866
## 3 N10156  115966
## 4 N102UW   25722
## 5 N103US   24619
## 6 N104UW   25157
## 7 N10575  150194
## 8 N105UW   23618
## 9 N107US   21677
## 10 N108UW   32070
## # i 4,034 more rows

```

Arithmetic and recycling rules

- When the vector lengths are mismatched, the elements of the shorter vector is *recycled* or repeated:

```

x <- c(1, 2, 10, 20)
x / 5

```

```
## [1] 0.2 0.4 2.0 4.0
```

```
x / c(5, 5, 5, 5)
```

```
## [1] 0.2 0.4 2.0 4.0
```

- R throws a warning when the length of the longer vector is not a multiple of the length of the shorter vector:

```
x * c(1, 2)
```

```
## [1] 1 4 10 40
```

```
x * c(1, 2, 3)
```

```
## Warning in x * c(1, 2, 3): longer object length is not a multiple of shorter
## object length
```

```
## [1] 1 4 30 20
```

- Recycling rules are also applied to logical comparisons.

Minimum and Maximum

- The functions `pmin()` and `pmax()` return the smallest or largest value in each row:

```
df <- tribble(
  ~x, ~y,
  1, 3,
  5, 2,
  7, NA
)

df |>
  mutate(
    min = pmin(x, y, na.rm = TRUE),
    max = pmax(x, y, na.rm = TRUE)
  )
```

```
## # A tibble: 3 x 4
##       x     y   min   max
##   <dbl> <dbl> <dbl> <dbl>
## 1     1     3     1     3
## 2     5     2     2     5
## 3     7    NA     7     7
```

Modular arithmetic

- The function `%/%` gives the integer part of the quotient and `%%` gives the remainder:

```
1:10 %/% 3
```

```
## [1] 0 0 1 1 1 2 2 2 3 3
```

```
1:10 %% 3
```

```
## [1] 1 2 0 1 2 0 1 2 0 1
```

- Modular arithmetic is handy for unpacking time into `hour` and `minute`:

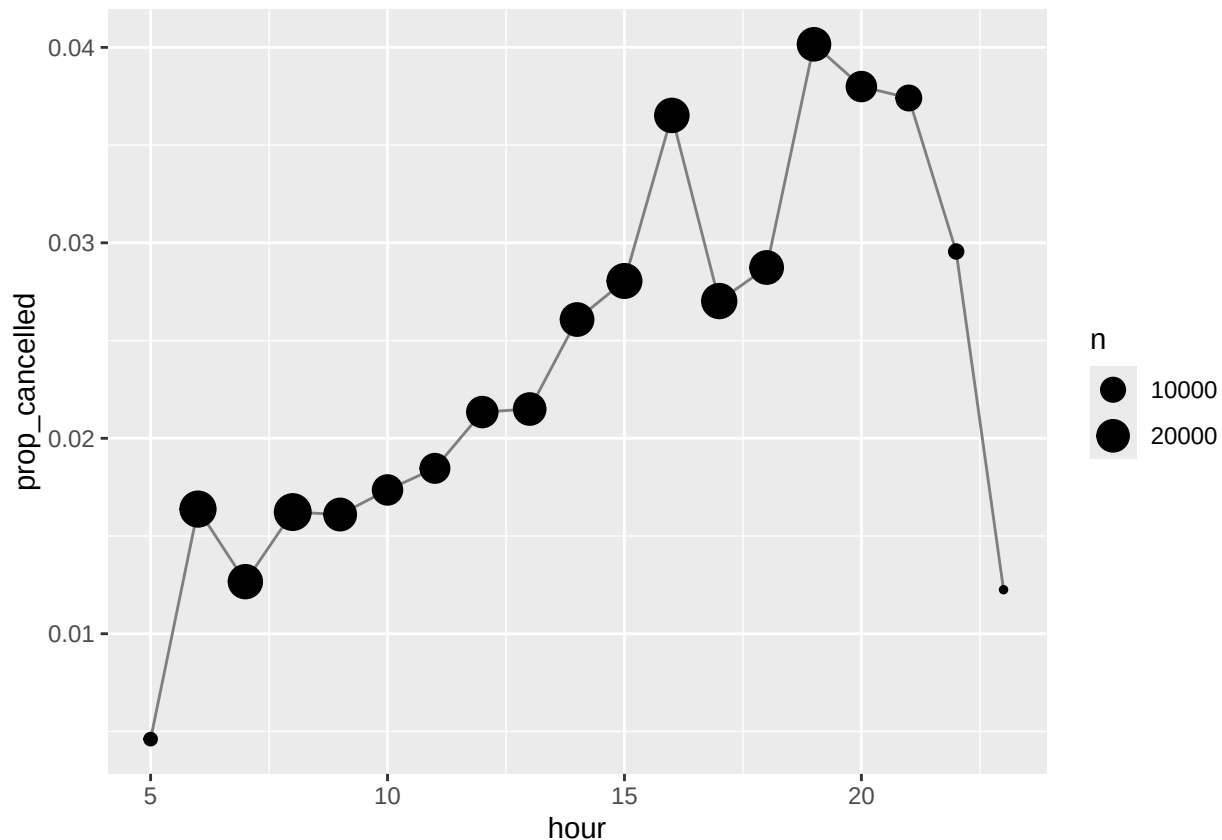
```
flights |>
  mutate(
    hour = sched_dep_time %/% 100,
    minute = sched_dep_time %% 100,
    .keep = "used"
  )
```

```
## # A tibble: 336,776 x 3
##   sched_dep_time hour minute
##         <int> <dbl> <dbl>
## 1           515     5     15
## 2           529     5     29
```

```
## 3          540      5      40
## 4          545      5      45
## 5          600      6        0
## 6          558      5      58
## 7          600      6        0
## 8          600      6        0
## 9          600      6        0
## 10         600      6        0
## # i 336,766 more rows
```

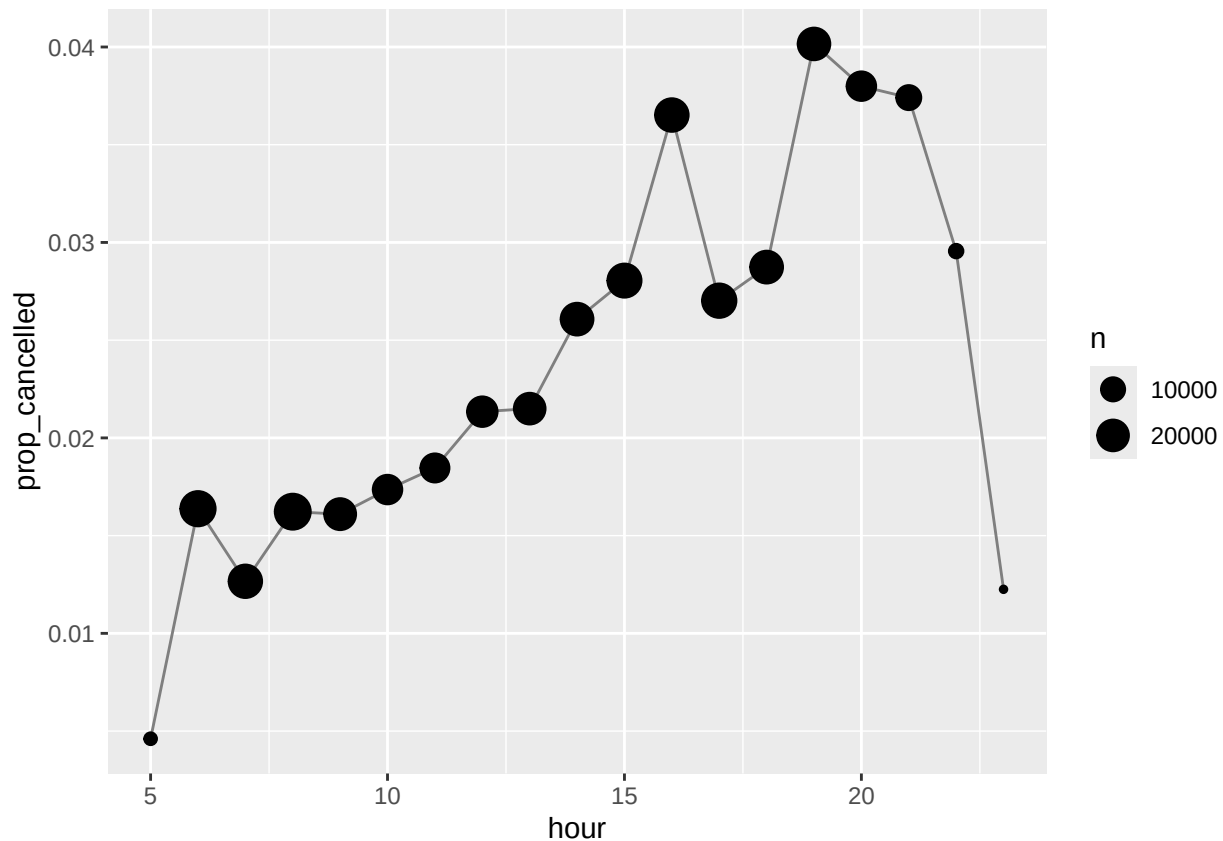
- From section 12, we learned to determine the proportion of TRUE values to FALSE values in a logical vector by using `mean()`.
- Combining this with modular arithmetic, we can plot the proportion of cancelled flights against the hour:

```
flights |>
  group_by(hour = sched_dep_time %/% 100) |>
  summarize(
    prop_cancelled = mean(is.na(dep_time)),
    n = n()
  ) |>
  filter(hour > 1) |>
  ggplot(aes(x = hour, y = prop_cancelled)) +
  geom_line(color = "grey50") +
  geom_point(aes(size = n))
```



This is a condensed version of the code below:

```
flights |>
  mutate(hour = sched_dep_time %/% 100) |>
  group_by(hour) |>
  summarize(
    prop_cancelled = mean(is.na(dep_time)),
    n = n()
  ) |>
  filter(hour > 1) |>
  ggplot(aes(x = hour, y = prop_cancelled)) +
  geom_line(color = "grey50") +
  geom_point(aes(size = n))
```



Logarithms

- Useful for dealing with data that ranges across multiple orders of magnitude.
- `log2()` and `log10()` are generally easier to interpret than logs of other bases.
- In base 2, every one step “forward” corresponds to doubling on the original scale, and every step “backward” corresponds to halving – think of the f-stops in photography.

Rounding

- Rounding to the nearest integer:

```
round(123.456)
```

```
## [1] 123
```


- Changing the precision of rounding using the `digits` argument:

```
round(123.456, digits = 2) # two digits to the right of decimal point
```

```
## [1] 123.46
```

```
round(123.456, 1) # one digit...
```

```
## [1] 123.5
```

```
round(123.456, -1) # nearest ten
```

```
## [1] 120
```

```
round(123.456, -2) # nearest hundred
```

```
## [1] 100
```

- R uses a rounding scheme called “Banker’s rounding” – a number halfway between two integers will be rounded to the even integer.

```
round(c(1.5, 2.5))
```

```
## [1] 2 2
```

- Because of this rounding scheme, we need to use `floor()` or `ceiling()` if we want to specifically round up or round down:

```
x <- 123.456
```

```
floor(x)
```

```
## [1] 123
```

```
ceiling(x)
```

```
## [1] 124
```

- Since `floor()` and `ceiling()` don’t have a `digits` argument, here’s a trick you can use to control the number of digits used in rounding:

```
# Round down to the nearest two digits
```

```
floor(x / 0.01) * 0.01
```

```
## [1] 123.45
```

```
# Round up to the nearest two digits
```

```
ceiling(x / 0.01) * 0.01
```

```
## [1] 123.46
```

Cutting numbers into ranges

- Using `cut()` to break up a numeric vector into discrete buckets or bins:

```
x <- c(1, 2, 5, 10, 15, 20)
```

```
# With even spacing
```

```
cut(x, breaks = c(0, 5, 10, 15, 20))
```

```
## [1] (0,5] (0,5] (0,5] (5,10] (10,15] (15,20]
```

```
## Levels: (0,5] (5,10] (10,15] (15,20]
```

```
# With uneven spacing
cut(x, breaks = c(0, 5, 10, 100))

## [1] (0,5] (0,5] (0,5] (5,10] (10,100] (10,100]
## Levels: (0,5] (5,10] (10,100]

# breaks with custom labels
cut(x,
    breaks = c(0, 5, 10, 15, 20),
    labels = c("small", "medium", "large", "extra_large")
)

## [1] small      small      small      medium     large      extra_large
## Levels: small medium large extra_large

• Any values outside the range of breaks will become NA

y <- c(NA, -10, 5, 10, 30)
cut(y, breaks = c(0, 5, 10, 15, 20))

## [1] <NA> <NA> (0,5] (5,10] <NA>
## Levels: (0,5] (5,10] (10,15] (15,20]
```

Cummulative and rolling aggregates

- Base R provides functions that start with cum:

```
x <- 1:10
cumsum(x)

## [1] 1 3 6 10 15 21 28 36 45 55

s <- 1:100
cumsum(s)

## [1] 1 3 6 10 15 21 28 36 45 55 66 78 91 105 120
## [16] 136 153 171 190 210 231 253 276 300 325 351 378 406 435 465
## [31] 496 528 561 595 630 666 703 741 780 820 861 903 946 990 1035
## [46] 1081 1128 1176 1225 1275 1326 1378 1431 1485 1540 1596 1653 1711 1770 1830
## [61] 1891 1953 2016 2080 2145 2211 2278 2346 2415 2485 2556 2628 2701 2775 2850
## [76] 2926 3003 3081 3160 3240 3321 3403 3486 3570 3655 3741 3828 3916 4005 4095
## [91] 4186 4278 4371 4465 4560 4656 4753 4851 4950 5050
```

This will make Gauss smile for sure.

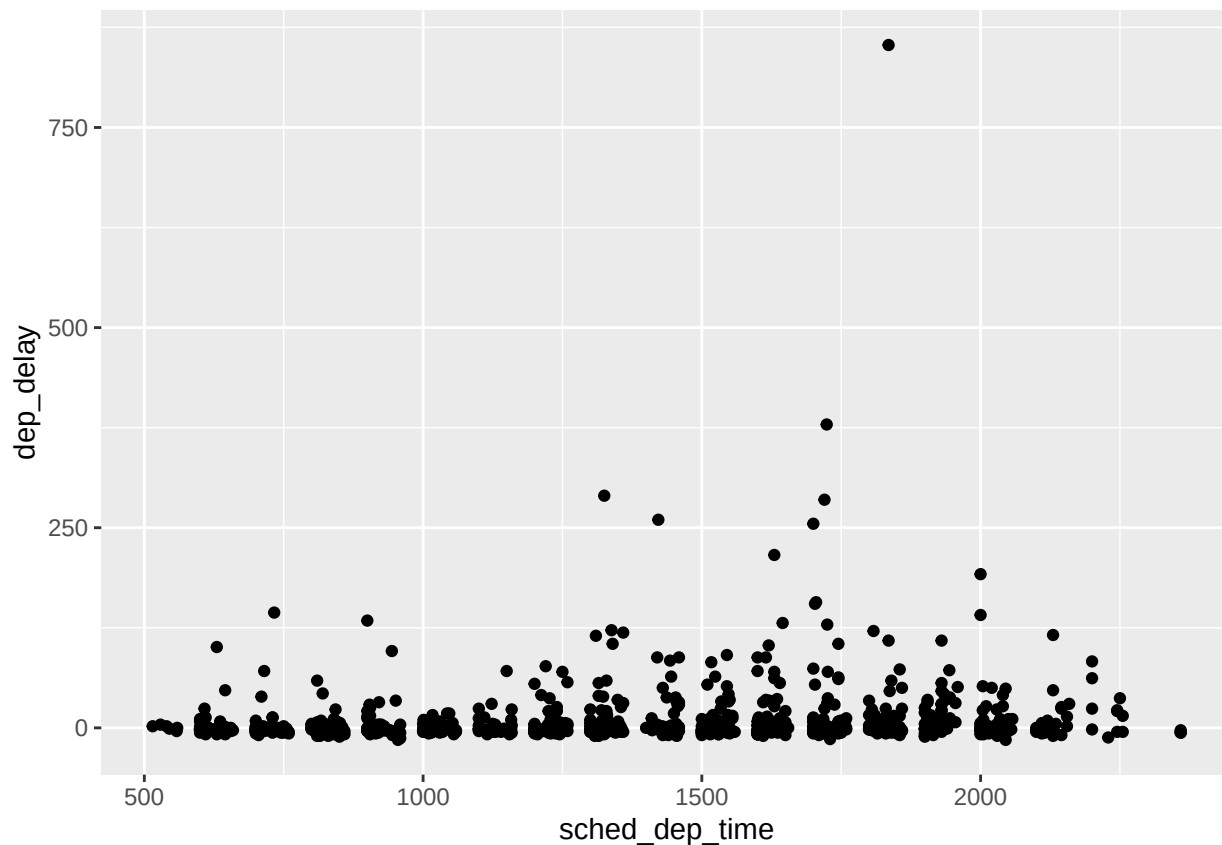
- More functions for rolling or sliding aggregates are available in the `slider` package.

Exercises

3. Currently `dep_time` and `sched_time` are convenient to look at, but hard to compute with because they're not really continuous numbers. You can see the basic problem by running the code below: there's a gap between each hour.

```
flights |>
  filter(month == 1, day == 1) |>
  ggplot(aes(x = sched_dep_time, y = dep_delay)) +
  geom_point()

## Warning: Removed 4 rows containing missing values or values outside the scale range
## (`geom_point()`).
```

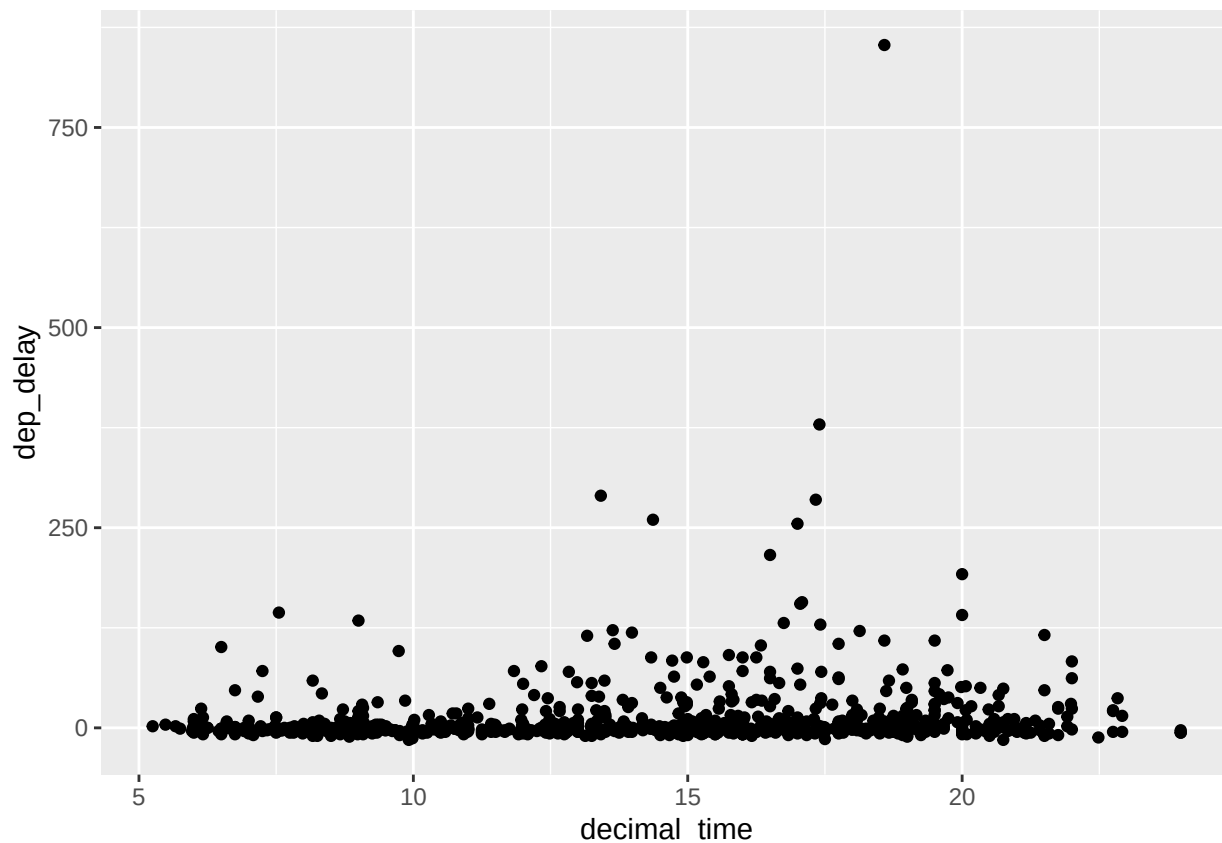


Convert them to a more truthful representation of time (either fractional hours or minutes since midnight).

Ans.

```
flights |>
  mutate(
    hour = sched_dep_time %/% 100,
    minute = sched_dep_time %/% 100,
    decimal_time = hour + minute / 60
  ) |>
  filter(month == 1, day == 1) |>
  ggplot(aes(x = decimal_time, y = dep_delay)) +
  geom_point()
```

```
## Warning: Removed 4 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



4. Round dep_time and arr_time to the nearest five minutes.

Ans.

```
flights |>
  mutate(
    dep_hour = dep_time %/% 100,
    dep_minute = sched_dep_time %/% 100,
    dep_minute = round(dep_minute / 5) * 5,
    dep_hour = if_else(dep_minute == 60, dep_hour + 1, dep_hour),
    dep_minute = if_else(dep_minute == 60, 0, dep_minute),
    .after = dep_time
  )
```

```
## # A tibble: 336,776 x 21
##   year month   day dep_time dep_hour dep_minute sched_dep_time dep_delay
##   <int> <int> <int>   <int>    <dbl>    <dbl>         <int>    <dbl>
## 1  2013     1     1     517         5         15           515         2
## 2  2013     1     1     533         5         30           529         4
## 3  2013     1     1     542         5         40           540         2
## 4  2013     1     1     544         5         45           545        -1
## 5  2013     1     1     554         5          0           600        -6
## 6  2013     1     1     554         6          0           558        -4
## 7  2013     1     1     555         5          0           600        -5
## 8  2013     1     1     557         5          0           600        -3
## 9  2013     1     1     557         5          0           600        -3
## 10 2013     1     1     558         5          0           600        -2
## # i 336,766 more rows
```

```
## # i 13 more variables: arr_time <int>, sched_arr_time <int>, arr_delay <dbl>,
## #   carrier <chr>, flight <int>, tailnum <chr>, origin <chr>, dest <chr>,
## #   air_time <dbl>, distance <dbl>, hour <dbl>, minute <dbl>, time_hour <dtm>
# Similar approach for arr_time.
```

General transformations

Ranks

- `min_rank()` uses the typical method for dealing with ties:

```
x <- c(1, 2, 2, 3, 4, NA)
min_rank(x)
```

```
## [1] 1 2 2 4 5 NA
```

- Using `desc()` to rank from highest to lowest:

```
min_rank(desc(x))
```

```
## [1] 5 3 3 2 1 NA
```

- Examples of other ranking functions:

```
df <- tibble(x = x)
df |>
  mutate(
    row_number = row_number(x),
    dense_rank = dense_rank(x),
    percent_rank = percent_rank(x),
    cume_dist = cume_dist(x)
  )
```

```
## # A tibble: 6 x 5
##       x row_number dense_rank percent_rank cume_dist
##   <dbl>     <int>     <int>         <dbl>     <dbl>
## 1     1         1         1           0         0.2
## 2     2         2         2          0.25        0.6
## 3     2         3         2          0.25        0.6
## 4     3         4         3          0.75        0.8
## 5     4         5         4           1           1
## 6    NA        NA        NA           NA          NA
```

Important arguments for `rank()` functions: - `ties.method`: for breaking ties - `na.last = "keep::"` to keep NAs as NA

- `row_number()`, when used without any arguments will give the “current” row.
- We can take advantage of this when we need to divide data into similarly sized groups:

```
df <- tibble(id = 1:10)
df |>
  mutate(
    row0 = row_number() - 1,
    three_groups = row0 %% 3,
    three_in_each_group = row0 %/% 3
  )
```

```
## # A tibble: 10 x 4
##       id row0 three_groups three_in_each_group
##   <int> <dbl>       <dbl>         <dbl>
## 1     1     0         0             0
## 2     2     1         1             0
## 3     3     2         2             0
## 4     4     3         0             1
## 5     5     4         1             1
## 6     6     5         2             1
## 7     7     6         0             2
## 8     8     7         1             2
## 9     9     8         2             2
## 10    10     9         0             3
```

Offsets

- Using `lead()` and `lag()`:

```
x <- c(2, 5, 11, 11, 19, 35)
```

```
lag(x)
```

```
## [1] NA  2  5 11 11 19
```

```
lead(x)
```

```
## [1]  5 11 11 19 35 NA
```

Clever uses for `lag()`:

```
# Get the difference between current and previous value
x - lag(x)
```

```
## [1] NA  3  6  0  8 16
```

```
# Determine if the current value changes
x == lag(x)
```

```
## [1]    NA FALSE FALSE  TRUE FALSE FALSE
```

Consecutive identifiers

- Useful when we want the items to be in a different category after crossing a certain threshold.
- Example: A situation where a new session should begin after a gap of more than x minutes since last activity:

```
events <- tibble(
  time = c(0, 1, 2, 3, 5, 10, 12, 15, 17, 19, 20, 27, 28, 30)
)
```

```
events <- events |>
  mutate(
    diff = time - lag(time, default = first(time)),
    has_gap = diff >= 5
  )
```

```
events
```

```
## # A tibble: 14 x 3
##   time  diff has_gap
##   <dbl> <dbl> <lgl>
## 1     0     0 FALSE
## 2     1     1 FALSE
## 3     2     1 FALSE
## 4     3     1 FALSE
## 5     5     2 FALSE
## 6    10     5  TRUE
## 7    12     2 FALSE
## 8    15     3 FALSE
## 9    17     2 FALSE
## 10   19     2 FALSE
## 11   20     1 FALSE
## 12   27     7  TRUE
## 13   28     1 FALSE
## 14   30     2 FALSE
```

- Example: Similar situation as before, but we want to get numeric results so that we can apply other functions:

```
events |> mutate(
  group = cumsum(has_gap)
)
```

```
## # A tibble: 14 x 4
##   time  diff has_gap group
##   <dbl> <dbl> <lgl>   <int>
## 1     0     0 FALSE     0
## 2     1     1 FALSE     0
## 3     2     1 FALSE     0
## 4     3     1 FALSE     0
## 5     5     2 FALSE     0
## 6    10     5  TRUE      1
## 7    12     2 FALSE     1
## 8    15     3 FALSE     1
## 9    17     2 FALSE     1
## 10   19     2 FALSE     1
## 11   20     1 FALSE     1
## 12   27     7  TRUE      2
## 13   28     1 FALSE     2
## 14   30     2 FALSE     2
```

- Example: Create a new grouping every time one of its arguments changes using `consecutive_id()` and `slice_head()`:

```
df <- tibble(
  x = c("a", "a", "a", "b", "c", "c", "d", "e", "a", "a", "b", "b"),
  y = c(1, 2, 3, 2, 4, 1, 3, 9, 4, 8, 10, 199)
)

df |>
  group_by(id = consecutive_id(x)) |>
  slice_head(n = 1)
```

```
## # A tibble: 7 x 3
## # Groups:   id [7]
```

```
##   x      y   id
##   <chr> <dbl> <int>
## 1 a      1    1
## 2 b      2    2
## 3 c      4    3
## 4 d      3    4
## 5 e      9    5
## 6 a      4    6
## 7 b     10    7
```

Exercises

1. Find the 10 most delayed flights using a ranking function.
2. Which plane (**tailnum**) has the worst on-time record?
3. What time of day should you fly if you want to avoid delays as much as possible?
4. What does the code do?

```
flights |>
  group_by(dest) |>
  filter(row_number() < 4)
```

```
## # A tibble: 311 x 19
## # Groups:   dest [105]
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
## 1  2013     1     1     517           515           2     830           819
## 2  2013     1     1     533           529           4     850           830
## 3  2013     1     1     542           540           2     923           850
## 4  2013     1     1     544           545          -1    1004          1022
## 5  2013     1     1     554           600          -6     812           837
## 6  2013     1     1     554           558          -4     740           728
## 7  2013     1     1     555           600          -5     913           854
## 8  2013     1     1     557           600          -3     709           723
## 9  2013     1     1     557           600          -3     838           846
## 10 2013     1     1     558           600          -2     753           745
## # i 301 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

Ans. It returns only the first three rows of each group created by `group_by(dest)`. Approximately equal to 105 groups x 3.

What does this code do?

```
flights |>
  group_by(dest) |>
  filter(row_number(dep_delay) < 4)
```

```
## # A tibble: 310 x 19
## # Groups:   dest [104]
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
## 1  2013     1     1     848           851          -3    1155          1136
## 2  2013     1     1     946           959         -13    1146          1202
## 3  2013     1     4    2140          2159        -19    2241          2316
## 4  2013     1     5     859           901          -2    1144          1201
```



```
## 5 2013      1      5      2143      2159      -16      2257      2315
## 6 2013      1      7      1801      1810       -9      2116      2142
## 7 2013      1      7      2013      2030      -17      2150      2206
## 8 2013      1      8       735       750      -15       912       940
## 9 2013      1      9      1847      1858      -11      2024      2055
## 10 2013     1     10       810       827      -17       955      1031
## # i 300 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

Ans. It returns only the first three rows of `dep_delay` for each group created by `group_by(dest)`.

5. For each destination, compute the total minutes of delay. For each flight, compute the proportion of the total delay for its destination.

Ans.

Total minutes of delay for each destination:

```
flights |>
  filter(arr_delay > 0) |>
  group_by(dest) |>
  summarize(
    total_arr_delay = sum(arr_delay, na.rm = TRUE)
  )
```

```
## # A tibble: 103 x 2
##   dest total_arr_delay
##   <chr>          <dbl>
## 1 ABQ           4487
## 2 ACK           2974
## 3 ALB           9580
## 4 ANC            62
## 5 ATL        300299
## 6 AUS        39940
## 7 AVL          3671
## 8 BDL          6953
## 9 BGR          6940
## 10 BHM         7267
## # i 93 more rows
```

Proportion of total delay per destination for each flight:

```
flights |>
  filter(arr_delay > 0) |>
  group_by(tailnum, dest) |>
  summarize(
    ave_prop_arr_delay = mean(arr_delay / sum(arr_delay)),
    .groups = "drop"
  )
```

```
## # A tibble: 34,458 x 3
##   tailnum dest ave_prop_arr_delay
##   <chr>   <chr>          <dbl>
## 1 D942DN ATL           1
## 2 D942DN MCO           0.5
## 3 NOEGMQ ATL           0.0417
```

```
## 4 NOEGMQ BNA 0.0476
## 5 NOEGMQ CLE 0.167
## 6 NOEGMQ CLT 0.0370
## 7 NOEGMQ CMH 0.125
## 8 NOEGMQ DCA 0.143
## 9 NOEGMQ DTW 0.25
## 10 NOEGMQ MSP 0.05
## # i 34,448 more rows
```

6. Delays are typically temporally correlated: even once the problem that caused the initial delay has been resolved, later flights are delayed to allow earlier flights to leave. Using `lag()`, explore how the average flight delay for an hour is related to the average delay for the previous hour.

```
flights |>
  mutate(hour = dep_time %/% 100) |>
  group_by(year, month, day, hour) |>
  summarize(
    dep_delay = mean(dep_delay, na.rm = TRUE),
    n = n(),
    .groups = "drop"
  ) |>
  mutate(lag = dep_delay - lag(dep_delay))
```

```
## # A tibble: 7,996 x 7
##   year month   day hour dep_delay    n    lag
##   <int> <int> <int> <dbl>   <dbl> <int> <dbl>
## 1  2013     1     1     5   -1.53    17  NA
## 2  2013     1     1     6   -1.16    51  0.373
## 3  2013     1     1     7    1.38    37  2.54
## 4  2013     1     1     8   14.1    64 12.7
## 5  2013     1     1     9    6.02    52 -8.09
## 6  2013     1     1    10    0.513   39 -5.51
## 7  2013     1     1    11    4.98    45  4.46
## 8  2013     1     1    12    2.14    43 -2.84
## 9  2013     1     1    13    9.17    47  7.03
## 10 2013     1     1    14    4.32    50 -4.85
## # i 7,986 more rows
```

7. Look at each destination. Can you find flights that are suspiciously fast (i.e. flights that represent a potential data entry error)? Compute the air time of a flight relative to the shortest flight to that destination. Which flights were most delayed in the air?

Ans. Based on speed

```
flights |>
  mutate(
    speed = distance / (air_time / 60)
  ) |>
  select(carrier, flight, dest, air_time, distance, speed) |>
  arrange(desc(speed))
```

```
## # A tibble: 336,776 x 6
##   carrier flight dest air_time distance speed
##   <chr>   <int> <chr>   <dbl>   <dbl> <dbl>
## 1 DL      1499 ATL      65      762  703.
## 2 EV      4667 MSP      93     1008  650.
## 3 EV      4292 GSP      55      594  648
```

```
## 4 EV      3805 BNA      70      748 641.
## 5 DL      1902 PBI      105     1035 591.
## 6 DL       315 SJU      170     1598 564
## 7 B6       707 SJU      172     1598 557.
## 8 AA       936 STT      175     1623 556.
## 9 DL       347 SJU      173     1598 554.
## 10 B6      1503 SJU      173     1598 554.
## # i 336,766 more rows
```

Since the ave. speed of commercial airlines, is usually between 500 to 600 mph, flights that are suspiciously fast or suspiciously slow can be flagged.

We can also do this by getting the standard deviation of `air_time`. Flights whose air time exceed the standard deviation by a certain threshold can be flagged.

Data entry checking based on air time of flight relative to the shortest flight:

```
# Showing the top n flights which took longer than the shortest air time
flights |>
  select(carrier, flight, dest, air_time, distance) |>
  group_by(flight, dest) |>
  mutate(
    shortest_air_time = if_else(air_time == min(air_time), air_time, min(air_time)),
    prop_to_shortest = air_time / shortest_air_time,
  ) |>
  arrange(desc(prop_to_shortest)) |>
  head(10)
```

```
## # A tibble: 10 x 7
## # Groups:   flight, dest [10]
##   carrier flight dest  air_time distance shortest_air_time prop_to_shortest
##   <chr>   <int> <chr>    <dbl>    <dbl>          <dbl>          <dbl>
## 1 B6      1491 ACK      141      199           35           4.03
## 2 AA      1750 BOS       96      187           32           3
## 3 UA      1248 BOS       92      200           32          2.88
## 4 UA       541 BOS       81      200           33          2.45
## 5 DL      1164 BOS       73      187           30          2.43
## 6 EV      4304 IAD       88      212           37          2.38
## 7 US      1860 PHL       75       96           32          2.34
## 8 UA      1240 BOS       82      200           35          2.34
## 9 B6      1012 BOS       77      187           33          2.33
## 10 UA       321 BOS       71      200           32          2.22
```

8. Find all destinations that are flown by at least two carriers. Use those destinations to come up with a relative ranking of carriers based on their performance for the same destination.

Ans.

```
# ranking carriers based on least amount of delays
flights |>
  select(carrier, dest, dep_delay) |>
  group_by(dest, carrier) |>
  mutate(
    n = n()
  ) |>
  filter(n >= 2) |>
  summarize(
    ave_dep_delay = mean(dep_delay, na.rm = TRUE)
```

```

) |>
mutate(
  ranking = rank(ave_dep_delay)
) |>
arrange(dest, ranking) |>
# show the top 4 per destination
filter(row_number() < 5)

```

`summarise()` has grouped output by 'dest'. You can override using the
`.groups` argument.

```

## # A tibble: 262 x 4
## # Groups:   dest [103]
##   dest carrier ave_dep_delay ranking
##   <chr> <chr>         <dbl>   <dbl>
## 1 ABQ    B6             13.7     1
## 2 ACK    B6              6.46     1
## 3 ALB    EV             23.6     1
## 4 ANC    UA             12.9     1
## 5 ATL    9E              0.965    1
## 6 ATL    WN              2.34     2
## 7 ATL    MQ              9.35     3
## 8 ATL    DL             10.4     4
## 9 AUS    WN              3.84     1
## 10 AUS   DL             10.8     2
## # i 252 more rows

```

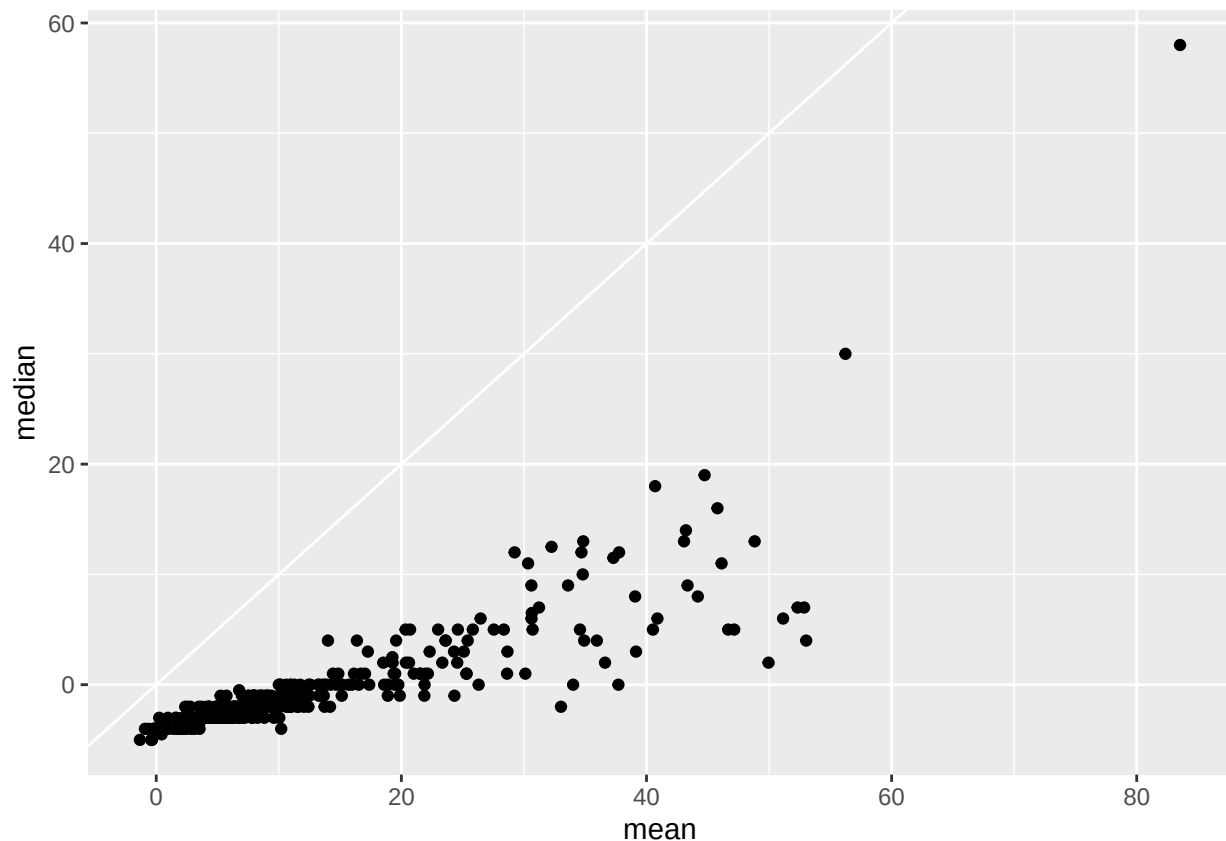
Numeric summaries

Measures of central tendencies

```

flights |>
  group_by(year, month, day) |>
  summarize(
    mean = mean(dep_delay, na.rm = TRUE),
    median = median(dep_delay, na.rm = TRUE),
    n = n(),
    .groups = "drop"
  ) |>
  ggplot(aes(x = mean, y = median)) +
  geom_abline(slope = 1, intercept = 0, color = "white") +
  geom_point()

```



- There is no **mode** function in R; this is not useful in real-world data and statisticians tend not to use this function.

Minimum, maximum, and quantiles

Looking at the the 95% quantile of a data feature while ignoring the extreme 5% of cases:

```
flights |>
  group_by(year, month, day) |>
  summarize(
    max = max(dep_delay, na.rm = TRUE),
    q95 = quantile(dep_delay, 0.95, na.rm = TRUE),
    .groups = "drop"
  )
```

```
## # A tibble: 365 x 5
##   year month   day    max   q95
##   <int> <int> <int> <dbl> <dbl>
## 1  2013     1     1   853  70.1
## 2  2013     1     2   379   85
## 3  2013     1     3   291   68
## 4  2013     1     4   288   60
## 5  2013     1     5   327   41
## 6  2013     1     6   202   51
## 7  2013     1     7   366  51.6
## 8  2013     1     8   188  35.3
## 9  2013     1     9  1301  27.2
## 10 2013     1    10  1126   31
```

```
## # i 355 more rows
```

Spread

- `IQR()` is basically `quantile(x, 0.75) - quantile(x, 0.25)`.

Exploring the curious case of EGE Airport using IQR:

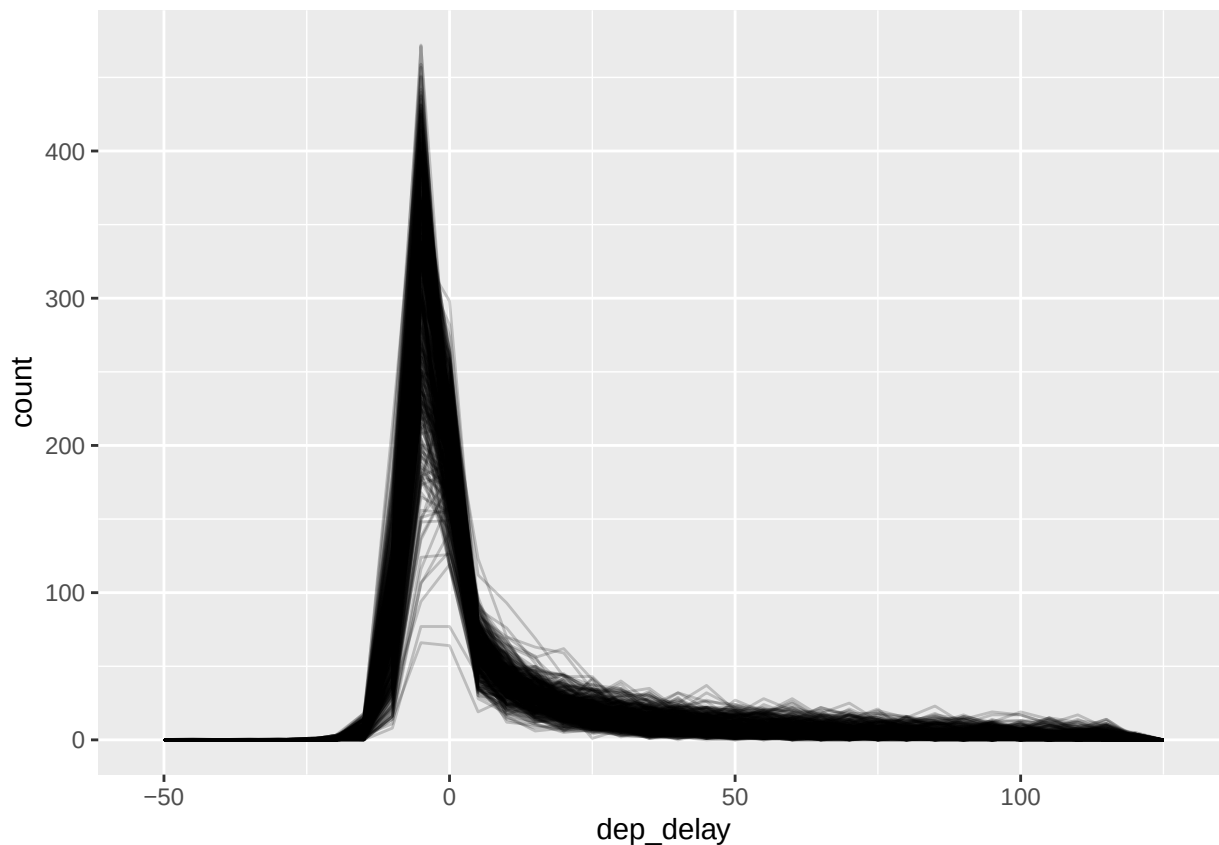
```
flights |>
  group_by(origin, dest) |>
  summarize(
    distance_iqr = IQR(distance),
    n = n(),
    .groups = "drop"
  ) |>
  filter(distance_iqr > 0)
```

```
## # A tibble: 2 x 4
##   origin dest distance_iqr      n
##   <chr>  <chr>         <dbl> <int>
## 1 EWR    EGE             1    110
## 2 JFK    EGE             1    103
```

Distributions

- Frequency plots of `dep_delay` for each day, for 365 days, overlaid, used to check if the distributions for subgroups resemble the whole:

```
flights |>
  filter(dep_delay < 120) |>
  ggplot(aes(x = dep_delay, group = interaction(day, month))) +
  geom_freqpoly(binwidth = 5, alpha = 1/5)
```



- Whenever creating numerical summaries, it is a good idea to include the number of observations in each group.

Extracting values at nth position

- Example for extracting the nth value using position functions `first`, `last`, and `nth`:

```
flights |>
  group_by(year, month, day) |>
  summarize(
    first_dep = first(dep_time, na_rm = TRUE),
    fifth_dep = nth(dep_time, 5, na_rm = TRUE),
    last_dep = last(dep_time, na_rm = TRUE)
  )
```

`summarise()` has grouped output by 'year', 'month'. You can override using the
`.groups` argument.

```
## # A tibble: 365 x 6
## # Groups:   year, month [12]
##   year month   day first_dep fifth_dep last_dep
##   <int> <int> <int>    <int>    <int>    <int>
## 1  2013     1     1      517      554    2356
## 2  2013     1     2       42      535    2354
## 3  2013     1     3       32      520    2349
## 4  2013     1     4       25      531    2358
## 5  2013     1     5       14      534    2357
## 6  2013     1     6       16      555    2355
## 7  2013     1     7       49      536    2359
```

```
## 8 2013 1 8 454 544 2351
## 9 2013 1 9 2 524 2252
## 10 2013 1 10 3 530 2320
## # i 355 more rows
```

- Be careful when using `first`, `nth`, and `last` – the argument for removing NAs is `na_rm` and *not* `na.rm`.
- Extracting values at positions using filtering on ranks:

```
flights |>
  group_by(year, month, day) |>
  mutate(r = min_rank(sched_dep_time)) |>
  filter(r %in% c(1, max(r)))

## # A tibble: 1,195 x 20
## # Groups:   year, month, day [365]
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
## 1 2013     1     1     517           515           2     830           819
## 2 2013     1     1    2353          2359          -6     425           445
## 3 2013     1     1    2353          2359          -6     418           442
## 4 2013     1     1    2356          2359          -3     425           437
## 5 2013     1     2      42          2359          43     518           442
## 6 2013     1     2     458           500          -2     703           650
## 7 2013     1     2    2351          2359          -8     427           445
## 8 2013     1     2    2354          2359          -5     413           437
## 9 2013     1     3      32          2359          33     504           442
## 10 2013     1     3     235          2359         156     700           437
## # i 1,185 more rows
## # i 12 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>, r <int>
```

Summary functions using `mutate()`

- Useful when a standardized summary function is preferred.

Examples of `mutate` commands for summarizing data:

- `x / sum(x)` for getting the proportion of total.
- `(x - mean(x)) / sd(x)` for calculating the z-score (standardized to have a mean of 0 and sd of 1).
- `(x - min(x)) / (max(x) - min(x))` for standardizing the data range to `[0, 1]`.
- `x / first(x)` for generating an index based on the first observation.